



Basic Docker Commands & Running the *Hello World* Container – Creative Theory Notes

1. Verifying Docker Using CLI

The first step in working with Docker is verifying that it is properly installed. This is done using the **terminal or command prompt** to check the Docker version.

- If the version is displayed → Docker is installed and working
- If an error appears → Docker is either not installed or not configured correctly

This confirmation ensures the system is ready to run Docker commands.

2. Introduction to the *Hello World* Docker Image

Docker provides a very small and lightweight image called “**hello-world**”, commonly used for beginners.

Key characteristics:

- Official Docker image
- Written in **C programming language**
- Extremely lightweight
- Designed only to test Docker installation and workflow

Running this image confirms that Docker can pull images, create containers, and execute them successfully.

3. Running an Image and the Pull Process

When the user runs the *hello-world* image for the first time, Docker checks whether the image exists **locally**.

- If the image is not found locally:
 - Docker automatically looks for it on **Docker Hub**
 - Uses the default tag “**latest**”
 - Pulls the image from the remote repository

This demonstrates Docker’s ability to seamlessly fetch missing images from Docker Hub.

4. Docker Hub as a Remote Repository

Docker Hub acts as a centralized cloud repository where Docker images are stored.

Features of Docker Hub:

- Hosts official images like **Nginx, MongoDB, NodeJS, Ubuntu**
- Allows users to search for images
- Supports pulling and pushing images
- Provides verified and official images for reliability

The *hello-world* image is confirmed to be an **official and trusted image**, making it ideal for learning.

5. Containers Creation and Auto-Naming

Once the *hello-world* image is pulled and run:

- Docker creates a **container**
- The container executes its task and exits immediately
- Docker assigns a **random container name** (e.g., *nice_bog*) if no name is specified

This highlights that:

- Image → Blueprint
- Container → Running (or executed) instance

Naming containers becomes important in real-world, multi-container environments.

6. Listing Containers Using Docker Commands

Docker provides commands to view containers:

- **Running containers only**
 - Displays active containers
- **All containers (running + stopped)**
 - Shows container ID
 - Image used
 - Creation time
 - Exit status
 - Container name

The **-a** flag is crucial for viewing stopped containers, especially when managing or deleting resources.

7. Listing Docker Images

Docker images stored locally can be viewed using a dedicated command.

Image listing shows:

- Repository name
- Image tag (version)
- Image ID
- Creation time
- Image size

This helps users track downloaded images and manage disk space.

8. Removing Containers and Images

Docker enforces a dependency rule between containers and images.

- **Removing containers:**
 - Stopped containers can be removed using container ID or name
- **Removing images:**
 - Images **cannot be removed** if a container (running or stopped) depends on them
 - Containers must be deleted first

This ensures system stability and prevents accidental data loss.

9. Exploring Docker Help Command

Docker provides a built-in **help system** to explore available commands.

Commonly used Docker commands include:

- **run** – Create and start a container
- **exec** – Run commands inside a container
- **ps** – List containers
- **build** – Build an image
- **pull** – Download an image
- **push** – Upload an image
- **images** – List images
- **login** – Authenticate with Docker Hub
- **search** – Search images on Docker Hub
- **rm** – Remove containers
- **rmi** – Remove images

These commands form the foundation of Docker usage.

10. Understanding Image vs Container Naming

A common beginner mistake is confusing **image names** with **container names**.

Important clarification:

- Images have names (e.g., `hello-world`)
- Containers have **IDs and names**
- Containers are started using **container ID or container name**, not image name

Docker generates long container IDs, but only the first few characters are required to reference them.

11. Manual Container Lifecycle (Step-by-Step)

Instead of using a single command, Docker allows manual control over the container lifecycle:

1. **Search image** – Confirms image availability on Docker Hub
2. **Pull image** – Downloads the image locally
3. **Create container** – Creates a container from the image
4. **Start container** – Runs the container
5. **Stop / Pause container** – Controls execution
6. **Remove container** – Deletes the container

This step-by-step process helps learners understand what happens internally.

12. Stop vs Pause Commands

Docker provides different execution control commands:

- **Stop:**
 - Gracefully stops a running container
- **Pause:**
 - Temporarily freezes a running container
 - Only works if the container is currently running
 - Fails if the container is already stopped

These commands help manage container behavior efficiently.

13. The Power of `docker run`

The `docker run` command is a **shortcut command** that performs multiple actions at once:

- Pulls the image (if not present)
- Creates a container
- Starts the container
- Runs the application

This makes `docker run` the most commonly used Docker command for quick execution.

14. Conclusion

This section introduces the **core Docker commands** required to:

- Verify Docker installation
- Pull images from Docker Hub
- Create, start, stop, and remove containers
- Manage Docker images and system resources

Running the *hello-world* container confirms that Docker is properly installed and functioning. These commands form the foundation for understanding Docker architecture and advanced container workflows.

If you want next, I can:

- Make a **Docker commands cheat sheet**
- Create **step-by-step lab exercises**
- Add **real-world examples**
- Summarize this into **short exam answers**
- Move on to **Docker architecture theory** 🚀